

Discussion 8

Note: Your TA will probably not cover all the problems on this worksheet. The discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, discussions, and homework.

This Week's Cool AI Demo/Video:

Anduril EagleEye: <https://www.youtube.com/watch?v=x9B02pFKpJo>

Guess who (temporarily) tanked googles market cap by 100B with one tweet: https://www.youtube.com/watch?v=Ej6hnsQgV_c

Decode neural signals with AI: <https://www.youtube.com/watch?v=PLfddDWy7eE>

modular robots: <https://www.youtube.com/watch?v=GbL3mjoZt0A>

1 Backprop in Practice: Staged Computation

We will apply backpropagation for the function $f(x, y, z) = (x + y)z$.

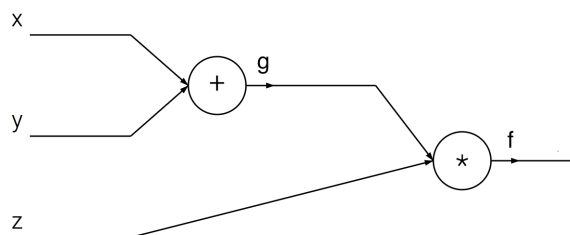
- (a) Decompose f into two simpler functions.

Solution:

We can conveniently express f as: $g = x + y$, $f = gz$. The original $f(x, y, z)$ expression is simple enough to differentiate directly, but this particular approach of thinking about it in terms of its decomposition can help with understanding the intuition behind backprop.

- (b) Draw the computation graph that represents f following the composition from part (a).

Solution:



- (c) Write the steps of the forward pass and backward pass through the network.

Solution:

Forward pass:

$$g = x + y,$$

$$f = g \cdot z.$$

Backward pass:

Backprop through $f = gz$:

$$\frac{\partial f}{\partial z} = g, \quad \frac{\partial f}{\partial g} = z.$$

Backprop through $g = x + y$ (using the chain rule):

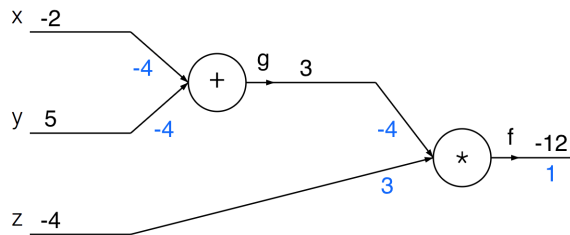
$$\frac{\partial f}{\partial x} = \frac{\partial g}{\partial x} \cdot \frac{\partial f}{\partial g} = 1 \cdot \frac{\partial f}{\partial g} = \frac{\partial f}{\partial g},$$

$$\frac{\partial f}{\partial y} = \frac{\partial g}{\partial y} \cdot \frac{\partial f}{\partial g} = 1 \cdot \frac{\partial f}{\partial g} = \frac{\partial f}{\partial g}.$$

- (d) Update your network drawing with the intermediate values in the forward and backward pass using the inputs $x = -2$, $y = 5$, and $z = -4$.

Solution:

We first run the forward pass (shown in black) of the network, which computes values from inputs to output. Then, we do the backward pass (blue), which starts at the end of the network and recursively applies the chain rule to compute the gradients.



Think of the gradients as flowing backwards through the network. By the time we reach the beginning of the network in backprop, each gate will have learned about the gradient of its output value on the final output of the entire circuit. Backprop can thus be thought of as message passing (via the gradient signal) between gates about whether they want their outputs to increase or decrease (and how strongly), so as to make the final output value higher.

2 A Two Layer Neural Network

Consider a neural network with two fully connected layers **but no nonlinearities**. The network maps input $\mathbf{x} \in \mathbb{R}^2$ to output $y \in \mathbb{R}$ according to the following functions:

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x}, \quad y = \mathbf{W}_2 \mathbf{h},$$

where $\mathbf{W}_1 \in \mathbb{R}^{2 \times 2}$ and $\mathbf{W}_2 \in \mathbb{R}^{1 \times 2}$. We define the loss as the mean squared error:

$$L = \frac{1}{2}(y - t)^2,$$

where t is the target output.

- (a) Write the steps of the forward pass and backward pass through the network. *Hint:* You may find the following vector calculus identities useful:

$$\begin{aligned} \text{(i)} \quad \frac{\partial}{\partial \mathbf{x}}(\mathbf{y}^\top \mathbf{x}) &= \mathbf{y}, \quad \text{(ii)} \quad \frac{\partial}{\partial \mathbf{A}}(\mathbf{y}^\top \mathbf{A} \mathbf{x}) = \mathbf{y} \mathbf{x}^\top, \quad \text{(iii)} \quad \frac{\partial}{\partial \mathbf{x}}(\mathbf{A} \mathbf{x}) = \mathbf{A}, \\ \text{(iv)} \quad \frac{\partial L}{\partial \mathbf{x}} &= \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{x}}, \quad \text{(v)} \quad \frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^\top \frac{\partial L}{\partial \mathbf{y}}, \quad \text{(vi)} \quad \frac{\partial L}{\partial \mathbf{A}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial \mathbf{A}} \end{aligned}$$

where y is a scalar, $\mathbf{x}$ and $\mathbf{y}$ are vectors, $\mathbf{A}$ is a matrix, and Y_{ij} is the ij th element of $\mathbf{Y}$.

Solution:

Forward pass:

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x}, \quad y = \mathbf{W}_2 \mathbf{h}, \quad L = \frac{1}{2}(y - t)^2$$

Backward pass:

Gradient of loss w.r.t output y :

$$\frac{\partial L}{\partial y} = y - t$$

Gradient of loss w.r.t weights $\mathbf{W}_2$ (using ii & iv):

$$\frac{\partial L}{\partial \mathbf{W}_2} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{W}_2} = (y - t) \mathbf{h}^\top$$

Gradient of loss w.r.t hidden units $\mathbf{h}$ (using i & iv):

$$\frac{\partial L}{\partial \mathbf{h}} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial \mathbf{h}} = (y - t) \mathbf{W}_2^\top$$

Gradient of loss w.r.t input $\mathbf{x}$ (using iii & v):

$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right)^\top \frac{\partial L}{\partial \mathbf{h}} = \mathbf{W}_1^\top ((y - t) \mathbf{W}_2^\top) = (y - t) \mathbf{W}_1^\top \mathbf{W}_2^\top$$

Gradient of loss w.r.t weights $\mathbf{W}_1$ (using ii & vi):

$$\frac{\partial L}{\partial \mathbf{W}_1} = \sum_{i=1}^2 \frac{\partial L}{\partial h_i} \frac{\partial h_i}{\partial \mathbf{W}_1} = \sum_{i=1}^2 \frac{\partial L}{\partial h_i} (\mathbf{e}_i \mathbf{x}^\top) = \left(\frac{\partial L}{\partial \mathbf{h}} \right) \mathbf{x}^\top = (y - t) \mathbf{W}_2^\top \mathbf{x}^\top$$

- (b) Write out y directly in terms of $\mathbf{x}$ and the weights $\mathbf{W}_1, \mathbf{W}_2$.

Solution:

We can substitute $\mathbf{h} = \mathbf{W}_1\mathbf{x}$ into $y = \mathbf{W}_2\mathbf{h}$:

$$y = \mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x}$$

Thus, the entire two-layer linear network is equivalent to a *single* linear transformation with effective weight matrix

- (c) Suppose you insert a nonlinearity $h(\cdot)$ after the first layer so that $\mathbf{h} = h(\mathbf{W}_1\mathbf{x})$. Why does this change what the network can represent? Illustrate using a simple example with ReLU.

Solution:

Without nonlinearities, the model can only represent *linear* mappings, no matter how many layers are stacked — multiple linear transformations collapse into one.

Adding a nonlinearity like ReLU introduces *piecewise linearity*:

$$\mathbf{h} = \max(0, \mathbf{W}_1\mathbf{x})$$

Now, the composition $\mathbf{W}_2h(\mathbf{W}_1\mathbf{x})$ can bend or change slope depending on the sign pattern of $\mathbf{W}_1\mathbf{x}$, enabling the network to approximate nonlinear functions.

For example, with ReLU, a 2-layer network can model hinge-like functions, while a linear network can only model straight lines.

3 Model Intuition

- (a) What can go wrong if you just initialize all the weights in a neural network to exactly zero? What about to the same nonzero value?

Solution:

Either of these neural networks will have an undesirable property: symmetry.

Perhaps the only property known [about initialization] with complete certainty is that the initial parameters need to “break symmetry” between different units. If two hidden units with the same activation function are connected to the same inputs, then these units must have different initial parameters. If they have the same initial parameters, then a deterministic learning algorithm applied to a deterministic cost and model will constantly update both of these units in the same way.

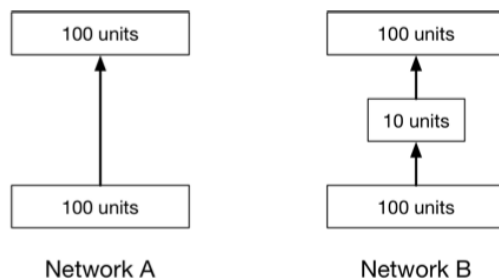
–*Deep Learning*, p. 301 (Goodfellow, Bengio, & Courville)

- (b) Adding nodes in the hidden layer gives the neural network more approximation ability, because you are adding more parameters. How many weight parameters are there in a neural network with architecture specified by $n = [n^{(0)}, n^{(1)}, \dots, n^{(\ell)}]$, a vector giving the number of nodes in each of the $\ell + 1$ layers? (Layer 0 is the input layer, and layer ℓ is the output layer.) Evaluate your formula for a network $n = [8, 10, 10, 3]$.

Solution:

The number of parameters for the connections between two layers is $n^{(i)}n^{(i+1)}$. If we are computing the number of weights and biases, there is an additional +1 in the formula, which becomes $(n^{(i)}+1)n^{(i+1)}$. The total number of weight parameters in the architecture is $\sum_{i=0}^{\ell-1} (n^{(i)}+1)n^{(i+1)}$. Applying this formula to the specified network gives $8 \times 10 + 10 \times 10 + 10 \times 3 = 210$ weights.

- (c) Consider the two networks in the image below, where the added layer in Network B has 10 units with **linear activation**. Give one advantage of Network A over Network B, and one advantage of Network B over Network A.



Solution:

Adding a linear-activated hidden layer does not make a network more powerful. It actually limits the functions that the network can represent. Therefore, A is more expressive than B. Specifically, it can learn any function that B can learn, plus some additional functions.

On the other hand, B is computationally cheaper because a matrix-vector product of size 10×100 followed by one of size 100×10 requires fewer operations than one of size 100×100 . B also has an embedding/bottleneck layer, which can be used in practice to create a compact representation of the data.